

AWS Networking: Complete Guide

From Physical Infrastructure to Advanced VPC Architecture

Chapter 1: The Physical Infrastructure Layer

1.1 Understanding AWS Global Infrastructure

Before writing a single line of application code or configuring any AWS service, every architect and developer must understand the physical foundation on which everything else is built. AWS operates a massive global network of data centers organized into a hierarchy of geographic constructs. Understanding this hierarchy is not merely academic. It directly influences the availability, performance, latency, compliance, and cost of every application you build on AWS.

The AWS global infrastructure can be thought of as a hierarchy moving from the largest geographic unit down to the smallest. At the top level, AWS divides the world into Regions. Within each Region, there are multiple Availability Zones. Extending from Regions, AWS has created Local Zones and Wavelength Zones for specialized low-latency use cases. Finally, AWS Outposts brings the AWS infrastructure physically into customer data centers. Each level of this hierarchy serves a distinct purpose, and understanding when and why to use each level is a core skill for any AWS professional.

1.2 AWS Regions

An AWS Region is a physical geographic area in the world where AWS has built and operates a cluster of data centers. Each Region is a completely self-contained unit of AWS

infrastructure. It has its own power grid connections, its own network infrastructure, its own set of AWS services, and its own independent operations team. The complete independence of Regions from one another is the most important characteristic to understand. If a catastrophic event affects one Region, every other Region continues operating normally without any impact.

As of the time of writing, AWS operates more than 30 geographic Regions worldwide, with additional Regions announced and under construction. These include regions in North America such as US East Northern Virginia, US East Ohio, US West Oregon, and US West Northern California. In South America, the São Paulo region serves customers across Latin America. In Europe, Regions include Ireland, Frankfurt, London, Paris, Milan, Stockholm, and Zurich. The Asia Pacific area has Regions in Singapore, Tokyo, Seoul, Mumbai, Sydney, Osaka, Hong Kong, Jakarta, and Hyderabad. The Middle East and Africa are served by Regions in Bahrain, UAE, and Cape Town.

Every AWS Region is identified by a code name. The US East Northern Virginia region has the code `us-east-1`. The Europe Ireland region has the code `eu-west-1`. The Asia Pacific Mumbai region has the code `ap-south-1`. When you configure AWS services, you always specify the region code to indicate where you want resources to be created.

Why Region Selection Matters

The decision of which Region to deploy your application in has profound implications that go far beyond simply picking a location on a map.

Latency is the first and most obvious consideration. Network latency is determined largely by the physical distance that data must travel. If your primary user base is in India, deploying your application in the `ap-south-1` Mumbai region will provide response times measured in single-digit milliseconds for those users. Deploying the same application in `us-east-1` Northern Virginia would result in response times of 200 to 300 milliseconds for the same Indian users, because the data must travel approximately 13,000 kilometers across the ocean floor and back. For most interactive applications, this difference is clearly perceptible to users and negatively impacts their experience.

Regulatory compliance is often the most constraining consideration. Many countries and industries have regulations governing where data can be stored and processed. The European Union's General Data Protection Regulation requires that personal data of EU citizens be stored and processed within the EU unless specific conditions are met. The healthcare industry in India requires patient data to remain within Indian borders. Financial services companies in certain countries must store transaction data domestically. These regulations can mandate your Region selection regardless of other considerations.

Service availability varies between Regions. AWS does not launch new services simultaneously in all Regions. A service might be available in us-east-1 for several months before it becomes available in ap-south-1. If your application depends on a specific AWS service, you must verify that the service is available in your chosen Region.

Pricing also varies between Regions. Running the same EC2 instance type in different Regions can cost between 10 and 30 percent more or less depending on the Region. Data transfer costs also vary by Region. For cost-sensitive workloads, comparing pricing across equivalent Regions can lead to meaningful savings.

Data Residency and Region Isolation

A critical architectural principle that flows from the Region model is that data does not automatically move between Regions. When you store data in a database in eu-west-1, that data stays in eu-west-1 unless you explicitly configure replication or migration to another Region. This is both a feature and a responsibility. It is a feature because it enables compliance with data residency requirements. It is a responsibility because it means you must explicitly design and implement any cross-Region redundancy or disaster recovery capabilities you need.

1.3 Availability Zones

Within each AWS Region, AWS operates multiple Availability Zones. An Availability Zone is one or more discrete data center facilities, each with redundant power, networking, and connectivity, housed in separate facilities within the Region's geographic area.

The physical separation of Availability Zones within a Region is deliberate and significant. AWS designs AZs so that they are far enough apart to be isolated from local disasters, such as fires, flooding, or power grid failures, while being close enough together to maintain high-speed, low-latency network connectivity between them. The fiber network links between Availability Zones within a Region provide bandwidth measured in terabits per second and latency typically under two milliseconds. This combination of isolation and high-speed connectivity is what makes multi-AZ architecture both safe and practical.

Each Availability Zone has a name consisting of the Region code followed by a letter. In the us-east-1 Region, the Availability Zones are named us-east-1a, us-east-1b, us-east-1c, us-east-1d, us-east-1e, and us-east-1f. Most Regions have three or more Availability Zones. AWS requires a minimum of three Availability Zones in each Region to provide sufficient redundancy options.

The Account-Level AZ Randomization

An important and often overlooked detail about Availability Zone naming is that the physical data center represented by a given AZ name differs between AWS accounts. What your AWS account refers to as us-east-1a may be a completely different physical data center than what another customer's account refers to as us-east-1a. AWS implemented this randomization to prevent all customers from preferring the first named AZ in each Region, which would cause uneven load distribution across the physical facilities.

This means that when coordinating with another AWS account, saying you are in us-east-1a is not precise. The correct way to identify a specific physical AZ when cross-account coordination is needed is to use the AZ ID, which is a stable identifier like use1-az1 that refers to the same physical facility regardless of which AWS account is viewing it.

Why Multiple Availability Zones Are Essential

The purpose of multiple Availability Zones is to enable high availability architectures that can survive the failure of an entire data center facility. AWS Availability Zones do experience outages. Several well-documented incidents have occurred where a single AZ experienced partial or complete loss of service for periods ranging from minutes to hours. Applications designed to run in a single AZ were affected by these incidents. Applications designed to run across multiple AZs continued operating normally throughout.

Consider a web application running on EC2 instances in a single Availability Zone. If that AZ experiences a power failure, the application becomes completely unavailable until the issue is resolved. If the same application runs with instances spread across three Availability Zones, a failure in one AZ affects only the instances in that AZ. The load balancer automatically detects the failure and stops routing traffic to the affected instances. The instances in the other two AZs continue serving traffic normally. Users might experience a brief period of increased latency as the load balancer redistributes traffic, but the application remains available.

For production workloads, deploying across at least two Availability Zones should be considered the minimum acceptable architecture. Deploying across three Availability Zones provides additional resilience and is recommended for critical applications. The cost of running resources across multiple AZs is the primary barrier, but for most production applications the improved availability justifies the additional cost.

1.4 Local Zones

AWS Local Zones are an extension of an AWS Region that places AWS compute, storage, database, and other select services closer to large population centers that are not near an existing AWS Region. Local Zones allow you to run latency-sensitive portions of your applications closer to end users, achieving single-digit millisecond latency for those users.

Local Zones are physically located in major metropolitan areas. For example, AWS has Local Zones in cities like Los Angeles, Dallas, Chicago, Atlanta, Boston, Denver, Houston, Miami, Minneapolis, New York City, Philadelphia, Phoenix, Seattle, and many others in the United States. Internationally, Local Zones exist in cities including Delhi, Kolkata, Copenhagen, Hamburg, Helsinki, Warsaw, and others.

A Local Zone is always an extension of a specific parent Region. The Los Angeles Local Zone is an extension of the us-west-2 Oregon Region. Resources in a Local Zone are part of the same VPC as resources in the parent Region. You can have an EC2 instance in the Los Angeles Local Zone and another EC2 instance in the us-west-2 Region communicating privately as if they were in the same network, because they are in the same VPC.

However, Local Zones do not support the full catalog of AWS services. They typically support EC2, EBS volumes, VPC networking, Application Load Balancers, and RDS. More specialized services may not be available. The idea is to run the latency-sensitive user-facing layer of your application in the Local Zone while the backend services that do not require ultra-low latency remain in the parent Region.

Local Zone Use Cases

Local Zones are designed for specific workload categories where sub-ten-millisecond latency to end users is a hard requirement.

Real-time gaming applications require extremely low latency between the game server and the player's device to provide a smooth, responsive gaming experience. A game server in a Local Zone close to a player cluster can provide the sub-five-millisecond latency that competitive gaming demands.

Live video streaming and media production workflows often require low-latency connections for real-time preview, editing, and collaboration. Content production companies with teams in major cities can benefit from Local Zones in those cities.

Augmented reality and virtual reality applications are highly sensitive to latency because any delay between user movement and the rendered response creates motion sickness and degrades the experience. Processing AR and VR workloads in a Local Zone minimizes this latency.

Financial trading applications, particularly those that involve algorithmic trading or market data processing, require the lowest possible latency to act on market events before competitors.

1.5 Wavelength Zones

AWS Wavelength Zones are an even more specialized infrastructure deployment designed to achieve the absolute minimum possible latency for applications serving users on 5G mobile networks. Wavelength Zones embed AWS compute and storage services directly within the 5G networks of telecommunications providers.

In a standard cloud architecture, a request from a 5G mobile device travels from the device to the nearest 5G tower, through the telecom provider's core network, out to the public internet, and then to an AWS data center. This path introduces latency from the telecom network's own infrastructure as well as the internet transit time. Even with a Local Zone, data still needs to leave the telecom network and travel to AWS infrastructure.

With a Wavelength Zone, AWS infrastructure is physically located inside the telecom provider's own network, at what is called the edge of the 5G network. A request from a 5G device travels to the nearest tower and then directly to the AWS compute resources within the telecom's network, without ever traversing the public internet. This architecture achieves single-digit millisecond, or even sub-millisecond, latency between 5G devices and application servers.

AWS has established Wavelength Zones in partnership with several major telecommunications providers. In the United States, Wavelength Zones exist within Verizon's network. In the United Kingdom and Germany, the partner is Vodafone. In Japan, the partner is KDDI. In South Korea, the partner is SK Telecom.

Wavelength Zones support a subset of AWS services, primarily EC2 for compute and EBS for storage. The Carrier Gateway, a specialized networking component discussed later in this guide, is used to route traffic between Wavelength Zones and the telecom's 5G network.

Wavelength Zones are relevant only for a narrow set of highly specialized use cases. These include connected autonomous vehicle systems that require real-time cloud processing, industrial IoT applications on 5G-connected factory floors, remote healthcare applications such as robotic surgery assistance over 5G, and consumer applications specifically designed around 5G ultra-low latency capabilities. The vast majority of applications have no need for Wavelength Zones.

1.6 AWS Outposts

AWS Outposts is a fully managed service that brings AWS infrastructure, services, APIs, and tools to virtually any on-premises location. With Outposts, the same AWS hardware that powers AWS data centers is physically installed in your own data center or co-location facility. AWS manages this hardware remotely, applying firmware updates, hardware maintenance, and monitoring.

Outposts come in two form factors. Outposts Racks are full 42U standard data center racks that can be configured with a variety of compute and storage capacity. They require dedicated data center space, power, and cooling in your facility. Outposts Servers are smaller 1U or 2U form factor servers designed for locations where a full rack is not feasible, such as retail stores, manufacturing floors, or remote facilities with limited space.

AWS staff are responsible for the initial installation and ongoing hardware maintenance of Outposts. You are responsible for providing the physical space, power, network connectivity, and physical security. The Outpost connects back to its parent AWS Region over a private network connection, which is required for control plane operations and for services that are not locally available on the Outpost.

Services available on Outposts include EC2, EBS, S3 on Outposts, RDS, EMR, ECS, EKS, and several others. The available services vary depending on the Outpost configuration. Services that are not available locally on the Outpost must be accessed over the connection back to the parent Region.

AWS Outposts Use Cases

AWS Outposts is designed for situations where the public cloud cannot be used due to specific constraints, but where the organization still wants to use AWS APIs, tools, and services for consistency.

Data sovereignty requirements represent the most common driver for Outposts. Some government agencies, defense contractors, financial institutions, and healthcare organizations operate under regulations that prohibit sensitive data from ever leaving their physical premises. With Outposts, these organizations can run AWS services locally while keeping all data within their own facility.

Ultra-low latency requirements that cannot be met even by Local Zones drive some Outposts deployments. A factory with robotic assembly lines controlled by computer vision requires processing latency measured in single-digit milliseconds between cameras and control systems. Even a Local Zone introduces some network latency that may be

unacceptable. With Outposts, compute resources are co-located with the physical equipment, providing sub-millisecond latency.

Disconnected or intermittently connected environments require local computing capability that functions independently of cloud connectivity. Remote mining operations, offshore oil platforms, ships at sea, and similar environments may have intermittent or very limited connectivity to the internet. Outposts can run workloads locally even when the connection to the parent Region is unavailable, synchronizing data when connectivity is restored.

Chapter 2: Virtual Private Cloud - Your Private Network in AWS

2.1 What is a Virtual Private Cloud?

A Virtual Private Cloud, universally abbreviated as VPC, is a logically isolated section of the AWS cloud where you can launch AWS resources in a virtual network that you define. A VPC gives you complete control over your virtual networking environment, including the selection of your own IP address range, the creation of subnets, the configuration of route tables, and the management of network gateways.

The concept of a VPC addresses a fundamental concern that organizations have when adopting cloud computing, which is the sharing of physical infrastructure with unknown parties. In the AWS cloud, thousands of different organizations run workloads on the same physical servers. Without logical isolation, this would be a serious security problem. A VPC provides that logical isolation. Even though your EC2 instances might be running on the same physical hardware as another customer's instances, the VPC ensures that there is a hard networking boundary between your resources and theirs. Your network traffic never crosses into their network, and their traffic never enters yours.

Think of the relationship between AWS and a VPC like the relationship between a large commercial building and individual office suites. The building is owned and maintained by the landlord, who is AWS. Within the building, each company has its own leased office suite, which is your VPC. The company can arrange the interior of its suite however it wants, deciding which rooms are accessible from the reception area and which require additional security clearance to enter. Employees from one company cannot walk into another company's suite. The building management handles the building's shared

infrastructure, such as elevators, electrical systems, and the internet connection to the building, but the interior of each suite is the responsibility of the tenant.

2.2 VPC Scope and Default VPC

A VPC is a Regional resource. This means a single VPC exists within one AWS Region and cannot span multiple Regions. However, within its Region, a VPC spans all Availability Zones. This is an important distinction. You do not create separate VPCs for each Availability Zone. You create one VPC for the Region, and then create subnets within that VPC that are placed in specific Availability Zones.

When you create a new AWS account, AWS automatically creates a default VPC in every Region. The default VPC is configured to be immediately usable for launching EC2 instances without any additional networking setup. It has a predefined CIDR block of 172.31.0.0/16, a public subnet in each Availability Zone, an internet gateway attached, and route tables configured to allow internet access. The default VPC is useful for getting started quickly and for experimentation, but it is not appropriate for production workloads because its open configuration does not follow security best practices.

For any production application, you should create a custom VPC with a carefully planned IP address range, a subnet structure appropriate for your architecture, and security controls designed for your specific requirements. Many organizations even delete the default VPC from all regions to prevent developers from accidentally using it for production resources.

2.3 CIDR Blocks - Defining Your IP Address Space

Every VPC is assigned an IP address range defined using Classless Inter-Domain Routing notation, universally abbreviated as CIDR. Understanding CIDR notation is fundamental to all VPC networking. Without this understanding, you cannot create subnets, configure routing, or design connectivity between networks.

An IP version 4 address is a 32-bit number written as four groups of decimal digits separated by periods. Each group, called an octet, represents 8 bits and can have a value from 0 to 255. The address 10.0.1.50 is an example. In binary, this address is 00001010.00000000.00000001.00110010.

CIDR notation adds a forward slash followed by a number to an IP address. This number, called the prefix length, specifies how many of the 32 bits in the address are the network portion, meaning they are fixed and identify the network. The remaining bits are the host portion, meaning they vary and identify individual devices within the network.

For the CIDR block 10.0.0.0/16, the first 16 bits are fixed and equal to 00001010.00000000. The remaining 16 bits can be anything, meaning any value from 0 to 65535. This gives the CIDR block 10.0.0.0/16 a range from 10.0.0.0 to 10.0.255.255, containing 65,536 addresses.

For the CIDR block 10.0.1.0/24, the first 24 bits are fixed and equal to 00001010.00000000.00000001. The remaining 8 bits can be anything from 0 to 255, giving the range 10.0.1.0 to 10.0.1.255, containing 256 addresses.

The relationship between prefix length and address count is inverse. A smaller prefix length means fewer fixed bits and more variable bits, resulting in a larger range of addresses. A larger prefix length means more fixed bits and fewer variable bits, resulting in a smaller range. The prefix length /8 gives approximately 16 million addresses. The prefix length /16 gives 65,536 addresses. The prefix length /24 gives 256 addresses. The prefix length /28 gives 16 addresses.

AWS reserves five IP addresses in every subnet. These reserved addresses are the first address in the range, which is the network address. The second address is reserved for the VPC router. The third address is reserved for AWS DNS. The fourth address is reserved for future use. The last address in the range is the broadcast address. For a /24 subnet with 256 addresses, five are reserved, leaving 251 usable addresses for your resources.

Choosing Your VPC CIDR

The choice of VPC CIDR block requires careful planning before you create the VPC. While it is possible to add secondary CIDR blocks to an existing VPC, you cannot change the primary CIDR block after creation. Choosing inadequately sized or conflicting address ranges upfront is a mistake that is painful to correct later.

AWS requires that VPC CIDR blocks use addresses from the private IP address ranges defined in RFC 1918. These ranges are 10.0.0.0 to 10.255.255.255 with a prefix of /8, 172.16.0.0 to 172.31.255.255 with a prefix of /12, and 192.168.0.0 to 192.168.255.255 with a prefix of /16. Within these ranges, you can choose any starting address and any prefix length between /16 and /28 for your VPC.

The most common choice for a production VPC is a /16 prefix, such as 10.0.0.0/16, which provides 65,536 addresses. This is large enough to accommodate many subnets across multiple Availability Zones while still leaving plenty of address space for growth.

The most important planning consideration when choosing your VPC CIDR is avoiding overlap with other networks you will need to connect to. If your VPC CIDR overlaps with your on-premises network CIDR or with another VPC's CIDR, you cannot create a direct network connection between them. A common enterprise pattern is to assign a unique /16 block to each VPC from a planned address space. For example, the production VPC might

use 10.0.0.0/16, the staging VPC might use 10.1.0.0/16, the development VPC might use 10.2.0.0/16, and the on-premises network might use 10.3.0.0/16. With this plan, all networks have unique, non-overlapping address spaces, and they can all be connected to each other.

IPv6 in VPCs

IPv4 addresses, with their 32-bit size, provide approximately 4.3 billion unique addresses. The exhaustion of the global IPv4 address pool has been a known problem for years and is the reason why obtaining new public IPv4 addresses is increasingly difficult and expensive. IPv6, with its 128-bit address size, provides approximately 340 undecillion unique addresses, effectively eliminating address exhaustion as a concern for the foreseeable future.

In AWS VPCs, IPv4 is always required and is the foundation of VPC networking. IPv6 is optional and can be enabled in addition to IPv4. When you enable IPv6 for a VPC, AWS assigns an IPv6 CIDR block from AWS-owned IPv6 address space. Unlike IPv4, where you choose your CIDR block, the IPv6 CIDR for a VPC is assigned by AWS and uses a /56 prefix, which provides 256 /64 subnets.

A critical difference between IPv4 and IPv6 in AWS VPCs is that all IPv6 addresses in AWS are globally unique and publicly routable. There is no concept of private IPv6 addresses in AWS. This means that an EC2 instance with an IPv6 address is potentially reachable from the internet on that address, subject to security group rules and routing configuration. This behavior is in contrast to IPv4, where private addresses like 10.0.x.x are not routable on the public internet.

2.4 Subnets - Dividing Your VPC

A subnet is a subdivision of your VPC's IP address space. You create subnets to organize your resources into groups based on their function and security requirements. Every EC2 instance, RDS database, and most other VPC resources must be placed in a subnet. The subnet determines the Availability Zone where the resource is physically located and provides the initial segmentation of traffic based on routing configuration.

Unlike a VPC, which spans all Availability Zones in a Region, each subnet exists within exactly one Availability Zone. A subnet cannot span multiple Availability Zones. This is why multi-AZ architectures require multiple subnets, one in each Availability Zone.

Public Subnets

A public subnet is a subnet that has been configured to allow direct communication with the internet. Technically, a subnet is public when it has a route to an Internet Gateway in its associated route table. This routing configuration is what distinguishes a public subnet from a private subnet. The terms public and private describe the connectivity characteristics of the subnet, not any inherent property of the subnet itself. You can convert a private subnet to a public subnet simply by adding a route to an internet gateway, and you can convert a public subnet to a private subnet by removing that route.

Resources in a public subnet can communicate with the internet in both directions, subject to security group rules. Inbound connections from the internet can reach resources in a public subnet if those resources have a public IP address or Elastic IP address and if the security group allows the inbound traffic. Outbound connections from resources in a public subnet can reach any internet destination.

The types of resources that typically belong in public subnets are those that need to accept inbound connections from the internet. Application Load Balancers belong in public subnets because they must receive HTTP and HTTPS traffic from users on the internet. NAT Gateways belong in public subnets because they need to reach the internet on behalf of resources in private subnets. Bastion hosts, which are jump servers used by administrators to access instances in private subnets via SSH, belong in public subnets because administrators connect to them from the internet.

Private Subnets

A private subnet is a subnet that does not have a direct route to an Internet Gateway. Resources in a private subnet cannot be reached directly from the internet, and they cannot make direct outbound connections to the internet. This isolation makes private subnets appropriate for resources that should not be exposed to the public internet.

Resources in private subnets can still communicate with the internet for outbound connections, such as downloading software updates or calling external APIs, through a NAT Gateway or NAT Instance located in a public subnet. They can also communicate with other AWS services without going through the internet by using VPC Endpoints. However, no device on the internet can initiate a connection to a resource in a private subnet, which provides significant protection against unauthorized access.

Application servers that process business logic should be in private subnets. Even though users interact with your application, they do so through the load balancer in the public subnet. The load balancer forwards requests to the application servers in private subnets. Users never need a direct path to the application servers. Database servers, cache clusters,

message queue workers, and any other backend infrastructure that does not need to accept connections from the internet should be in private subnets.

Subnet Sizing and Design Patterns

A common and effective subnet design for production applications uses at least three tiers of subnets across at least two Availability Zones. The public subnet tier hosts internet-facing resources like load balancers and NAT Gateways. The private subnet tier hosts application servers. The database subnet tier hosts data stores. This three-tier design with two or more AZs results in a minimum of six subnets.

When sizing subnets, consider the number of resources you plan to run in each subnet now and in the future. Because subnets cannot be resized after creation, it is better to size subnets slightly larger than your current needs to allow for growth. A common approach is to create /24 subnets, providing 251 usable addresses each, which is sufficient for most use cases while being manageable in size.

Chapter 3: Route Tables - Directing Network Traffic

3.1 What is a Route Table?

A route table is the component of VPC networking that determines where network traffic is directed. Every subnet in a VPC must be associated with exactly one route table, and that route table controls which network destinations are reachable from resources in that subnet. When a resource in a subnet sends a network packet, the VPC router consults the route table associated with the subnet to determine where to forward the packet.

A route table contains a collection of routes. Each route has two components. The destination is a CIDR block that specifies the range of IP addresses that the route applies to. The target is the AWS resource or connection that packets matching the destination should be sent to.

When a packet needs to be forwarded, the VPC router examines all routes in the route table and finds the routes whose destination range includes the packet's destination IP address. If multiple routes match, the router uses the longest prefix match rule, which

selects the most specific matching route. A route with destination 10.0.1.0/24 is more specific than a route with destination 10.0.0.0/16, so if both match a packet's destination, the /24 route wins. The route with destination 0.0.0.0/0, which means any IP address, matches every packet but is the least specific, so it only applies when no more specific route matches.

3.2 The Local Route

Every route table in a VPC automatically contains a local route. The local route has the destination set to the VPC's CIDR block and the target set to the keyword local. This route enables all resources within the VPC to communicate with each other using their private IP addresses, regardless of which subnet they are in.

The local route cannot be deleted or modified. It is permanently present in every route table in the VPC. This route is what allows an EC2 instance in a public subnet to communicate with an RDS database in a database subnet, even though they are in different subnets, as long as the security groups permit the traffic.

3.3 Internet Routes

Adding a route with the destination 0.0.0.0/0 and the target pointing to an Internet Gateway is what transforms a subnet from private to public. When a resource in this subnet needs to reach a destination that does not match any more specific route, it sends the packet to the Internet Gateway. Because 0.0.0.0/0 matches any IP address, all traffic destined for addresses outside the VPC is sent to the Internet Gateway, which forwards it to the internet.

For private subnets that need outbound-only internet access, the same 0.0.0.0/0 route is present but the target is a NAT Gateway rather than an Internet Gateway. The NAT Gateway allows outbound connections to initiate from the private subnet but prevents inbound connections from the internet from reaching the private subnet.

3.4 Main Route Table vs Custom Route Tables

When you create a VPC, AWS automatically creates a main route table. The main route table serves as the default route table for all subnets that have not been explicitly associated with a custom route table. Initially, the main route table contains only the local route.

Custom route tables are additional route tables that you create and explicitly associate with specific subnets. Custom route tables allow different subnets to have different routing configurations. Public subnets are associated with a custom route table that has a route to the Internet Gateway. Private subnets are associated with a different custom route table that has a route to the NAT Gateway. Database subnets might be associated with yet another custom route table that has only the local route.

The recommended best practice is to leave the main route table with only the local route and to create explicit custom route tables for each subnet tier. This approach ensures that new subnets created in the future do not accidentally inherit routing they should not have. If the main route table had a route to the Internet Gateway, any new subnet that was not explicitly associated with a custom route table would accidentally become a public subnet, potentially exposing resources unintentionally.

3.5 Route Priority and the Longest Prefix Match

Understanding how the VPC router selects among multiple matching routes is essential for designing networks that behave as expected.

The VPC router always selects the most specific matching route. Specificity is determined by the prefix length. A /32 route, which matches exactly one IP address, is the most specific possible. A /0 route, which matches all IP addresses, is the least specific. Between these extremes, a /24 route is more specific than a /16 route, which is more specific than an /8 route.

Consider a route table with the following routes. The first route has destination 10.0.0.0/16 and target local. The second route has destination 10.0.1.0/24 and target a specific peering connection. The third route has destination 0.0.0.0/0 and target a NAT Gateway.

For a packet destined for 10.0.1.50, both the /16 and /24 routes match. The /24 route is more specific, so the packet is sent to the peering connection. For a packet destined for 10.0.2.50, only the /16 route matches, so it is sent to local within the VPC. For a packet destined for 8.8.8.8, only the /0 route matches, so it is sent to the NAT Gateway.

Chapter 4: Internet Connectivity - Gateways and NAT

4.1 Internet Gateway

The Internet Gateway is the component that connects your VPC to the public internet. It is a horizontally scaled, redundant, and highly available VPC component that AWS manages. You do not need to worry about the Internet Gateway's capacity, availability, or maintenance. It automatically handles any amount of traffic without being a bottleneck.

Each VPC can have at most one Internet Gateway attached. You attach an Internet Gateway to a VPC as a simple configuration action. After attachment, you can reference the Internet Gateway as a route target in your route tables to enable internet connectivity for subnets.

The Internet Gateway performs a critical function called Network Address Translation, or NAT, for EC2 instances with public IP addresses. When an EC2 instance with a private IP address of 10.0.1.50 and a public IP address of 54.123.45.67 sends a packet to an internet destination, the packet's source address is 10.0.1.50. The Internet Gateway intercepts this packet and replaces the source address with the instance's public IP address, 54.123.45.67, before forwarding the packet to the internet. The Internet Gateway maintains a mapping of this translation. When the internet destination sends a response back to 54.123.45.67, the Internet Gateway receives the response, looks up the translation, replaces the destination address with 10.0.1.50, and forwards the packet to the correct instance within the VPC.

The public internet never sees private IP addresses. All communication between the internet and resources in your VPC occurs using public IP addresses. The Internet Gateway handles the translation between public and private addresses transparently.

For an EC2 instance to communicate with the internet through the Internet Gateway, three conditions must all be met. First, the instance's subnet must have a route to the Internet Gateway. Second, the instance must have either a public IPv4 address or an Elastic IP address. Third, the security group associated with the instance must allow the relevant inbound and outbound traffic.

4.2 Egress-Only Internet Gateway

The Egress-Only Internet Gateway is a specialized gateway designed specifically for IPv6 traffic. To understand why it exists, you need to understand a key difference between IPv4 and IPv6 in AWS.

For IPv4, private addresses like 10.0.x.x are not routable on the public internet. A packet with a source address of 10.0.1.50 would be dropped by internet routers because they do not know how to route to private addresses. This provides inherent protection because private IPv4 addresses cannot receive unsolicited inbound connections from the internet.

For IPv6 in AWS, all addresses are globally unique and publicly routable. An EC2 instance with an IPv6 address can theoretically receive inbound connections from any IPv6 host on the internet. If you want an EC2 instance to have outbound internet access for IPv6 traffic without being reachable from the internet for inbound IPv6 connections, you need the Egress-Only Internet Gateway.

The Egress-Only Internet Gateway allows outbound IPv6 traffic from instances in your VPC to the internet but prevents inbound IPv6 traffic from the internet from reaching your instances. It is stateful, meaning that if an instance initiates an outbound connection, the response traffic for that connection is allowed back in. But unsolicited inbound connections from internet hosts are blocked.

The Egress-Only Internet Gateway serves the same conceptual purpose for IPv6 that a NAT Gateway serves for IPv4, providing outbound internet access while preventing unsolicited inbound connections.

4.3 NAT Gateway

The NAT Gateway is one of the most commonly used components in production VPC architectures. It enables EC2 instances and other resources in private subnets to initiate outbound connections to the internet while preventing the internet from initiating inbound connections to those resources.

The operational need for a NAT Gateway arises from the combination of two architectural goals that are simultaneously desirable. The first goal is security, which is keeping application servers, databases, and other backend resources in private subnets where they cannot be directly reached from the internet. The second goal is functionality, which is allowing those same private resources to make outbound internet connections for practical operational needs such as downloading operating system updates, installing software packages, calling external APIs like payment processors or mapping services, and sending data to external monitoring services.

Without a NAT Gateway, resources in private subnets have no path to the internet at all. With a NAT Gateway, they can reach the internet for outbound traffic while remaining completely unreachable from the internet for inbound traffic.

How NAT Gateway Works

A NAT Gateway is always placed in a public subnet and is always associated with an Elastic IP address. The public subnet gives the NAT Gateway a path to the Internet Gateway and therefore to the internet. The Elastic IP address gives the NAT Gateway a stable, static public IP address that appears as the source address of all outbound traffic from the private subnets it serves.

When an EC2 instance in a private subnet with IP address 10.0.10.50 needs to reach an external API at IP address 54.100.200.50, the process works as follows. The instance creates a packet with source address 10.0.10.50 and destination address 54.100.200.50. The instance's route table has a route for 0.0.0.0/0 pointing to the NAT Gateway. The packet is forwarded to the NAT Gateway in the public subnet. The NAT Gateway replaces the source address 10.0.10.50 with its own Elastic IP address, for example 54.200.100.50, and creates an entry in its connection tracking table recording this translation. The NAT Gateway forwards the packet to the Internet Gateway, which forwards it to the internet. The external API processes the request and sends a response to 54.200.100.50. The Internet Gateway receives the response and forwards it to the NAT Gateway. The NAT Gateway looks up its connection tracking table and finds that 54.200.100.50 maps back to 10.0.10.50. The NAT Gateway replaces the destination address with 10.0.10.50 and forwards the response to the EC2 instance.

Throughout this entire process, the internet-facing API server communicates only with the NAT Gateway's Elastic IP address. It has no knowledge that the actual source of the request was 10.0.10.50. More importantly, nothing on the internet can initiate a connection to 10.0.10.50 because 10.0.10.50 is a private address with no route from the internet.

NAT Gateway Availability and High Availability

A NAT Gateway is an Availability Zone-scoped resource. A NAT Gateway placed in us-east-1a serves resources in private subnets in us-east-1a. If that Availability Zone experiences an outage, the NAT Gateway is unavailable along with all the resources in that AZ.

For a highly available architecture, you should create one NAT Gateway in each Availability Zone and configure the private subnets in each AZ to use the NAT Gateway in the same AZ. With this configuration, each AZ is self-contained for outbound internet access. If one AZ fails, the NAT Gateways in the other AZs continue operating normally for the resources that remain available in those AZs.

This configuration also avoids cross-AZ data transfer charges. When a resource in a private subnet in us-east-1a sends traffic through a NAT Gateway in us-east-1b, that traffic crosses Availability Zone boundaries. AWS charges for cross-AZ data transfer. By keeping each AZ's traffic within the AZ through local NAT Gateways, you avoid these charges.

A NAT Gateway is a managed AWS service, meaning AWS handles its scaling, availability, and maintenance within its Availability Zone. It can handle up to 45 gigabits per second of bandwidth and supports up to 55,000 simultaneous connections to a single destination. These limits are sufficient for virtually all application workloads.

NAT Gateway vs NAT Instance

Before AWS introduced the managed NAT Gateway service, organizations that needed NAT functionality for private subnets had to configure an EC2 instance to perform NAT manually. This approach, using a NAT Instance, is largely obsolete for production workloads but still appears in some legacy architectures and in cost-optimized development environments.

A NAT Instance is a regular EC2 instance configured to forward packets on behalf of other instances. It requires several specific configuration steps to function correctly. The EC2 instance must use an AMI specifically designed for NAT. The Source and Destination Check, a default EC2 behavior that drops packets not addressed to the instance itself, must be disabled. Security groups and routing must be configured. The NAT Instance must be in a public subnet with internet access.

The key operational difference between a NAT Gateway and a NAT Instance is that a NAT Instance requires ongoing management. You are responsible for patching the operating system, monitoring its health, and handling failures. If the NAT Instance fails, private subnets lose internet access until you replace it. A NAT Gateway, by contrast, is fully managed by AWS, scales automatically, and is highly available within its Availability Zone without any management effort on your part.

The Source and Destination Check mentioned above is a networking behavior worth understanding because it applies to all EC2 instances, not just NAT instances. By default, every EC2 instance performs a check on every network packet it receives. If a packet's source or destination address does not belong to that instance, the instance drops the packet. This behavior makes sense for normal instances because they should only send and receive traffic for themselves. However, for instances that are meant to forward traffic on behalf of other instances, such as NAT instances, VPN servers, or network appliance instances, this check must be disabled so the instance can handle packets addressed to other destinations.

Chapter 5: Connecting On-Premises Networks to AWS

5.1 The Hybrid Cloud Need

Most organizations migrating to AWS do not move everything to the cloud at once. The typical migration involves years of incremental movement, with some systems migrating to AWS while others remain on-premises. During this transition, and often permanently for certain workloads, you need secure, private connectivity between your on-premises networks and your AWS VPCs. AWS provides two primary mechanisms for this connectivity: AWS Site-to-Site VPN and AWS Direct Connect.

5.2 Customer Gateway and Virtual Private Gateway

Two AWS resources work together as the endpoints of a VPN connection between your premises and AWS.

The Customer Gateway is an AWS resource that represents your on-premises VPN device or software application. The Customer Gateway does not perform any networking function itself. It is a declarative resource in AWS that stores information about your VPN endpoint. When you create a Customer Gateway, you specify the public IP address of your on-premises VPN device. This is the IP address that AWS will use to establish the VPN tunnels.

The on-premises VPN device that a Customer Gateway represents can be a hardware router or firewall from vendors such as Cisco, Juniper, Palo Alto, Fortinet, or Check Point. It can also be a software-based VPN running on a Linux instance using software like OpenVPN or StrongSwan. What matters is that the device supports the IPsec protocol family and can maintain the VPN tunnel.

The Virtual Private Gateway is the AWS side of the VPN connection. It attaches to your VPC and serves as the AWS endpoint for VPN tunnels from your on-premises network. A Virtual Private Gateway is a highly available construct managed by AWS. Internally, AWS runs the Virtual Private Gateway across multiple physical systems in different Availability Zones to ensure that a hardware failure in one location does not take down your VPN connection.

5.3 AWS Site-to-Site VPN

AWS Site-to-Site VPN creates an encrypted IPsec connection between your on-premises network and your AWS VPC using the Customer Gateway and Virtual Private Gateway as the two endpoints.

Every Site-to-Site VPN connection consists of two separate tunnels, each connecting to a different AWS endpoint. AWS requires and creates two tunnels per VPN connection as a built-in redundancy mechanism. If one tunnel fails or the AWS endpoint it connects to experiences an issue, traffic can automatically fail over to the second tunnel, maintaining connectivity. AWS recommends using both tunnels simultaneously with BGP-based routing to achieve true active-active redundancy.

The tunnels use the IPsec protocol suite to provide encryption in transit. IPsec operates at the network layer and encrypts each IP packet before it is transmitted and decrypts it at the receiving end. The encryption protects the confidentiality and integrity of data traversing the tunnels. All traffic between your on-premises network and your VPC passes through these encrypted tunnels.

Each VPN tunnel supports up to 1.25 gigabits per second of bandwidth. This limitation is a consequence of the encryption and decryption processing required for IPsec. For applications requiring higher bandwidth, AWS Direct Connect or multiple VPN connections can be combined.

Routing for Site-to-Site VPN can be configured in two modes. Static routing requires you to manually specify the IP address ranges that should be routed through the VPN. Dynamic routing uses the BGP, which stands for Border Gateway Protocol, to automatically exchange routing information between your on-premises router and the Virtual Private Gateway. BGP-based dynamic routing is preferred because it automatically adapts to network topology changes, supports faster failover between the two tunnels, and eliminates the operational overhead of manually maintaining static routes.

After establishing a VPN connection, you must add routes to your VPC route tables to direct on-premises-destined traffic to the Virtual Private Gateway. You can enable route propagation on a route table, which causes the Virtual Private Gateway to automatically add the routes learned from your on-premises network via BGP to the route table, eliminating the need to manually manage these routes.

5.4 AWS Direct Connect

AWS Direct Connect is a networking service that provides a dedicated private network connection from your premises to AWS. Unlike Site-to-Site VPN, which uses the public

internet as the underlying transport, Direct Connect traffic never traverses the public internet. Instead, it travels over a private physical network connection that you establish between your facility and an AWS Direct Connect location.

AWS maintains Direct Connect locations at co-location facilities operated by companies like Equinix, CoreSite, and CyrusOne in cities around the world. At each Direct Connect location, AWS has installed its own network equipment. To establish a Direct Connect connection, you or your network service provider runs a physical cable from your router to the AWS router in the Direct Connect location. This physical connection, called a cross-connect, is the foundation of your Direct Connect circuit.

Direct Connect connections are available at several capacity options. Dedicated connections are physical connections reserved exclusively for your use and are available at 1 gigabit per second, 10 gigabits per second, and 100 gigabits per second. Hosted connections are physical connections operated by AWS Direct Connect partners that are partitioned and shared, available from 50 megabits per second up to 10 gigabits per second. Hosted connections are appropriate when your bandwidth requirement is less than 1 gigabit per second or when you want to avoid the cost and lead time of a dedicated connection.

Direct Connect Virtual Interfaces

A Direct Connect connection provides raw layer 2 connectivity between your network and AWS. To actually route traffic, you must create Virtual Interfaces, commonly called VIFs, on top of the physical connection.

A Private Virtual Interface connects your on-premises network to a specific VPC via the VPC's Virtual Private Gateway. Traffic routed through a Private VIF reaches resources in your VPC using their private IP addresses. A single Direct Connect connection can have multiple Private VIFs connecting to different VPCs.

A Public Virtual Interface connects your on-premises network to AWS public services like Amazon S3, Amazon DynamoDB, and other AWS services using their public endpoints. Traffic through a Public VIF does not enter any VPC. Instead, it reaches AWS public service endpoints through the AWS backbone network.

A Transit Virtual Interface connects your on-premises network to a Transit Gateway, which in turn provides access to multiple VPCs. Transit VIFs are the most scalable option for connecting large numbers of VPCs over Direct Connect.

Direct Connect Advantages Over VPN

Direct Connect provides several significant advantages over Site-to-Site VPN that justify its higher cost and longer setup time for appropriate use cases.

Performance consistency is the primary advantage. VPN over the public internet is subject to the variable congestion, routing changes, and quality fluctuations of the internet. Direct Connect traffic travels over a dedicated private circuit that is not shared with internet traffic. This provides consistent, predictable bandwidth and latency that you can rely on for performance-sensitive applications.

Higher bandwidth is available through Direct Connect than through VPN. A single Direct Connect connection supports up to 100 gigabits per second, compared to 1.25 gigabits per second per VPN tunnel. For workloads involving large data transfers, such as database migrations or video production workflows, Direct Connect's bandwidth advantage is often the deciding factor.

Lower latency is achievable with Direct Connect because traffic does not travel through the variable routing of the internet. The path from your facility to AWS through a dedicated circuit is shorter and more deterministic than the path through the internet.

Reduced data transfer costs apply when using Direct Connect for transferring large volumes of data into and out of AWS. The data transfer pricing for Direct Connect is lower than the internet data transfer pricing applied when using VPN or other internet-based connections for large volumes.

Direct Connect Limitations

Direct Connect requires physical infrastructure and time to set up. The process of ordering a cross-connect, coordinating with the colocation facility, provisioning the circuit, and establishing the connection typically takes weeks to months. This lead time is incompatible with urgent connectivity requirements.

Direct Connect is not encrypted by default. Unlike VPN, which uses IPsec encryption for all traffic, Direct Connect provides private but unencrypted connectivity. For compliance with regulations requiring encryption in transit, you can run a VPN connection over the Direct Connect circuit, which adds IPsec encryption while retaining the performance benefits of the dedicated physical connection.

Direct Connect is a single point of failure at the physical layer unless you implement redundancy. A single Direct Connect connection uses a single physical cable, and that cable can be cut or a port can fail. AWS recommends establishing two Direct Connect connections from two separate Direct Connect locations for critical workloads. Additionally,

maintaining a backup Site-to-Site VPN connection provides a lower-bandwidth failover path if all Direct Connect circuits are unavailable.

5.5 Direct Connect Gateway

The Direct Connect Gateway is a globally available resource that allows you to connect a single Direct Connect connection to multiple VPCs in different AWS Regions.

Without a Direct Connect Gateway, each Direct Connect connection can only connect to VPCs in the same Region. If you have VPCs in three Regions and you want all of them to connect to your on-premises network via Direct Connect, you would need three separate Direct Connect circuits, one in each Region. This is expensive and operationally complex.

With a Direct Connect Gateway, you establish one or two Direct Connect connections to a convenient Direct Connect location, then associate those connections with the Direct Connect Gateway. You then associate the Direct Connect Gateway with Virtual Private Gateways or Transit Gateways in multiple Regions. All associated VPCs in all associated Regions become accessible over the single Direct Connect connection through the Direct Connect Gateway.

An important limitation of Direct Connect Gateways is that they enable connectivity between your on-premises network and your VPCs but do not enable VPC-to-VPC connectivity. Two VPCs that are both connected to the same Direct Connect Gateway cannot communicate with each other through the Direct Connect Gateway. For VPC-to-VPC connectivity, you need VPC Peering or Transit Gateway.

Chapter 6: EC2 Instance Networking

6.1 Elastic Network Interfaces

An Elastic Network Interface, abbreviated as ENI, is the fundamental networking component for EC2 instances. It is a virtual network card that you can attach to EC2 instances. Every EC2 instance has at least one ENI, called the primary network interface, which is created automatically when the instance is launched. You can attach additional ENIs to an instance to provide it with multiple network interfaces.

An ENI is a resource in its own right in AWS. It has its own lifecycle that can be independent of any particular EC2 instance. You can create standalone ENIs, attach them to instances, detach them, and reattach them to different instances. This mobility makes ENIs useful for several advanced networking patterns.

Each ENI has a specific set of attributes. The ENI belongs to a specific subnet, which determines its Availability Zone and the pool of IP addresses from which it can be assigned addresses. The ENI has one or more security groups associated with it, and these security groups define the inbound and outbound traffic rules that apply to traffic through that ENI. The ENI has a MAC address, which is a hardware-level identifier that is fixed for the lifetime of the ENI.

Primary and Secondary Network Interfaces

When you launch an EC2 instance, the primary ENI is automatically created and attached. The primary ENI cannot be detached from the instance while the instance is running. It has a primary private IP address assigned from the subnet's CIDR range, and it is associated with the security groups you specify during launch. The primary ENI is always present and cannot be removed.

You can add secondary ENIs to an instance after launch. Secondary ENIs can be in different subnets from the primary ENI, as long as all subnets are in the same Availability Zone as the instance. Secondary ENIs can also have different security groups from the primary ENI. Secondary ENIs can be detached from an instance and reattached to a different instance.

The number of ENIs an instance can have depends on the instance type. Smaller instances support fewer ENIs. Larger instances support more. Each additional ENI also adds more secondary private IP addresses that can be assigned to the instance.

6.2 IP Address Types

An EC2 instance can have several different types of IP addresses, and understanding the differences between them is important for designing networks that behave correctly.

Primary Private IPv4 Address

Every EC2 instance has exactly one primary private IPv4 address assigned to its primary ENI. This address is selected from the CIDR range of the subnet in which the instance is launched. Either you specify the address explicitly during launch, or AWS automatically assigns an available address from the subnet's range.

The primary private IPv4 address is permanent for the life of the instance. It does not change when the instance is stopped and started. It does not change when the instance is rebooted. It is released only when the instance is terminated. This permanence makes the primary private IP address the stable, reliable identifier for the instance within the VPC.

Secondary Private IPv4 Addresses

In addition to the primary private IP address, an instance can have one or more secondary private IPv4 addresses assigned to its ENIs. The number of secondary addresses supported depends on the instance type.

Secondary private IP addresses are useful for hosting multiple applications or services on a single instance, where each application needs its own IP address. They are also useful for software licensing tied to IP addresses and for IP failover patterns where a secondary IP from a failed instance is moved to a healthy instance.

Secondary private IP addresses can be assigned to and removed from an ENI while the instance is running. They can also be moved from one ENI to another ENI on the same instance or to an ENI on a different instance.

Public IPv4 Address

A public IPv4 address is an internet-routable IP address that enables direct communication between an EC2 instance and the internet. Public IPv4 addresses are assigned automatically by AWS when an instance is launched in a subnet that has the auto-assign public IPv4 setting enabled, which is typically the case for public subnets.

Public IPv4 addresses are dynamic. They are assigned when the instance launches or starts and are released when the instance is stopped or terminated. Each time a stopped instance is restarted, it receives a new public IPv4 address from the pool. This dynamic nature means you cannot reliably reference a public IPv4 address in DNS records or in other configurations that require a stable value. For stable public addressing, you need an Elastic IP address.

Elastic IP Addresses

An Elastic IP address is a static, persistent public IPv4 address allocated to your AWS account. Unlike the dynamic public IPv4 addresses assigned automatically to instances, an Elastic IP address does not change. It belongs to your account until you explicitly release it.

The key capability of an Elastic IP address is that it can be associated with and disassociated from EC2 instances on demand. You can move an Elastic IP address from one instance to another in the same Region within seconds. This mobility enables failover

architectures where you quickly move a public IP from a failed instance to a replacement instance, allowing DNS records to continue pointing to the same IP address.

AWS charges for Elastic IP addresses that are allocated but not associated with a running instance. This policy discourages allocating Elastic IPs and leaving them unused, which would waste the scarce pool of public IPv4 addresses. When an Elastic IP is properly associated with a running instance, there is no hourly charge. Beginning in 2024, AWS introduced a charge for all public IPv4 addresses, including Elastic IPs associated with running instances, reflecting the genuine scarcity and cost of public IPv4 addresses.

Chapter 7: Advanced VPC Connectivity

7.1 VPC Peering

VPC Peering is a networking connection between two VPCs that enables resources in both VPCs to communicate with each other using private IP addresses, as if they were within the same network. Peering connections use AWS's private network backbone and never traverse the public internet, providing low latency and no bandwidth constraints.

VPC Peering can connect VPCs within the same AWS account, between different AWS accounts, and across different AWS Regions. Cross-Region peering, sometimes called inter-region peering, allows resources in VPCs in different geographic regions to communicate privately, which is valuable for global applications.

Establishing a Peering Connection

Creating a VPC Peering connection is a two-step process. First, the owner of one VPC sends a peering request to the target VPC. Second, the owner of the target VPC accepts the request. For peering between VPCs in different AWS accounts, the owners of both accounts must be involved. After the connection is accepted, both VPCs are connected, but traffic will not flow until routing is configured.

After establishing the peering connection, you must update the route tables in both VPCs. In the requester VPC, you add a route with the destination set to the acceptor VPC's CIDR and the target set to the peering connection identifier. In the acceptor VPC, you add a route with the destination set to the requester VPC's CIDR and the target set to the same peering connection identifier. Without both route table updates, traffic cannot flow

between the VPCs. You must also ensure that security groups in each VPC allow the relevant traffic from the other VPC's CIDR range.

VPC Peering Constraints

Non-overlapping CIDR blocks are a hard requirement for VPC Peering. If two VPCs have the same or overlapping IP address ranges, you cannot create a peering connection between them. This is why planning non-overlapping CIDR blocks across all VPCs in your environment is so important when you first design your address space.

VPC Peering is not transitive. This is a critical behavioral characteristic that surprises many users. If VPC A is peered with VPC B, and VPC B is peered with VPC C, VPC A cannot communicate with VPC C through VPC B. Each peering connection is a direct, bilateral connection between exactly two VPCs. Traffic from VPC A can reach VPC B through their direct peering connection. But from VPC B, the traffic cannot be forwarded on to VPC C, even though VPC B has a peering connection with VPC C.

If you need VPC A to communicate with VPC C, you must create a direct peering connection between VPC A and VPC C. In a network with many VPCs that all need to communicate with each other, this requirement leads to a large number of peering connections. For a network of ten VPCs that all need mutual connectivity, you need 45 peering connections. Managing this many connections and their associated route table entries becomes operationally complex, which is the primary motivation for using Transit Gateway in large environments.

7.2 AWS Transit Gateway

AWS Transit Gateway is a network transit hub that enables you to connect multiple VPCs and on-premises networks through a single, centralized gateway. Rather than creating a mesh of VPC Peering connections between every pair of VPCs, each VPC connects to the Transit Gateway once, and the Transit Gateway routes traffic between all connected networks.

The architectural model of Transit Gateway is called hub and spoke. The Transit Gateway is the hub, and all VPCs and on-premises networks are the spokes. Every spoke connects to the hub. Traffic between any two spokes flows through the hub. This topology dramatically reduces the number of connections needed as the number of networks grows.

Transit Gateway Attachments

An attachment is the mechanism by which a VPC or other network resource connects to a Transit Gateway. Different types of attachments exist for different resource types.

A VPC attachment connects a VPC to the Transit Gateway. When you create a VPC attachment, you specify which subnets in the VPC will have the attachment. AWS creates an ENI in each specified subnet, which serves as the entry and exit point for traffic flowing between the VPC and the Transit Gateway. Traffic from resources in the VPC that is destined for other VPCs attached to the Transit Gateway is routed to the Transit Gateway through these ENIs.

A VPN attachment connects a Site-to-Site VPN connection to the Transit Gateway. This allows on-premises networks connected via VPN to access all VPCs attached to the Transit Gateway through a single VPN connection.

A Direct Connect Gateway attachment connects a Direct Connect Gateway to the Transit Gateway. This allows on-premises networks connected via Direct Connect to access all VPCs attached to the Transit Gateway through a single Direct Connect circuit.

A peering attachment connects two Transit Gateways in different AWS Regions. This enables Transit Gateways in different Regions to exchange routes and route traffic between them, allowing truly global network architectures.

Transit Gateway Route Tables

Transit Gateway Route Tables determine how the Transit Gateway routes traffic between attachments. Unlike VPC route tables, which live in VPCs and route traffic entering or leaving a VPC, Transit Gateway Route Tables live within the Transit Gateway itself and control how the Transit Gateway forwards traffic from one attachment to another.

A Transit Gateway always has a default route table. By default, all attachments are associated with the default route table and propagate their routes to it. Under this default configuration, all attached VPCs and networks can communicate with each other through the Transit Gateway.

The real power of Transit Gateway Route Tables becomes apparent when you need network segmentation. By creating multiple route tables and carefully associating attachments with different route tables, you can control which networks can communicate with which other networks through the Transit Gateway.

Consider an organization with production VPCs, development VPCs, and a shared services VPC containing common resources. You create a production route table that knows about the production VPCs and the shared services VPC but not the development VPCs. You

create a development route table that knows about the development VPCs and the shared services VPC but not the production VPCs. The shared services VPC is associated with a route table that knows about all other VPCs. With this configuration, production workloads cannot reach development resources and development workloads cannot reach production resources, but both can reach the shared services. This is network-level isolation enforced through routing.

7.3 VPC Endpoints

VPC Endpoints allow resources within your VPC to communicate with supported AWS services without requiring traffic to leave the AWS network. Without VPC Endpoints, when an EC2 instance in a private subnet needs to access Amazon S3, the traffic must travel through a NAT Gateway, exit the VPC network, traverse the internet, and reach S3's public endpoint. With a VPC Endpoint, the traffic travels entirely within AWS's private network infrastructure, never leaving the AWS network backbone.

Gateway Endpoints

Gateway Endpoints are available for two specific AWS services, which are Amazon S3 and Amazon DynamoDB. They are implemented as entries in your VPC route table rather than as network interfaces. When you create a Gateway Endpoint for S3 in your VPC, AWS adds a route to the route tables you specify. The destination of this route is a prefix list, which is an AWS-managed list of IP address ranges used by S3 in the Region. The target of the route is the Gateway Endpoint identifier.

When an EC2 instance in a subnet with this route table entry sends a request to S3, the VPC router matches the destination IP address against the prefix list and routes the traffic to the Gateway Endpoint rather than to the NAT Gateway or Internet Gateway. The traffic travels within AWS's private network backbone to S3, without ever leaving the AWS network.

Gateway Endpoints have no associated cost. There is no hourly charge and no data processing charge. The only cost saved is the elimination of NAT Gateway data processing charges that would otherwise apply to this traffic. Gateway Endpoints also do not require the traffic to have a path to the internet. Even instances in database subnets with no internet route can access S3 and DynamoDB through Gateway Endpoints, which is useful for database backup and restore operations.

Gateway Endpoints work only within the VPC in which they are created. Resources in your on-premises network connected via VPN or Direct Connect cannot use a Gateway Endpoint

to access S3. For on-premises access to S3 via a private path, Interface Endpoints are required.

Interface Endpoints and AWS PrivateLink

Interface Endpoints are available for a much broader set of AWS services including EC2 API, CloudWatch, CloudWatch Logs, Systems Manager, Secrets Manager, KMS, SNS, SQS, API Gateway, and over a hundred other services. Interface Endpoints are also available for your own custom services and for third-party services offered through AWS Marketplace, using a feature called AWS PrivateLink.

An Interface Endpoint is implemented as an Elastic Network Interface in your subnet with a private IP address from that subnet's CIDR range. This ENI acts as a private entry point to the AWS service or third-party service. When DNS resolution is enabled for the endpoint, AWS overrides the DNS resolution for the service's endpoint hostname to return the private IP address of the Interface Endpoint ENI instead of the service's public IP address. This means that your application code does not need to change. It continues to call the service using its standard endpoint hostname, but DNS now resolves that hostname to a private IP address in your VPC.

Interface Endpoints support security groups. You can attach a security group to an Interface Endpoint to control which resources within your VPC can use the endpoint and on which ports. This provides an additional layer of access control beyond what is provided by IAM policies.

Interface Endpoints have an associated cost. AWS charges an hourly rate for each Interface Endpoint in each Availability Zone where it is created, plus a data processing charge per gigabyte of traffic through the endpoint. For highly regulated environments or high-traffic workloads, these costs are often justified by the security and compliance benefits of keeping traffic within the AWS network and by the elimination of NAT Gateway processing charges.

A significant advantage of Interface Endpoints over Gateway Endpoints is that Interface Endpoints are accessible from on-premises networks connected via Direct Connect or VPN. Because the Interface Endpoint has a private IP address in your VPC, and your on-premises network has private connectivity to your VPC, your on-premises systems can access AWS services through the Interface Endpoint without using the public internet. This is valuable for hybrid architectures where on-premises systems need to call AWS APIs or access services like S3 via a private network path.

Chapter 8: Carrier Gateway

8.1 What is a Carrier Gateway?

A Carrier Gateway is a specialized networking component used exclusively with AWS Wavelength Zones. Just as an Internet Gateway provides connectivity between a VPC and the public internet, a Carrier Gateway provides connectivity between a VPC in a Wavelength Zone and the carrier's telecommunications network.

When you deploy resources in a Wavelength Zone, those resources need two distinct network paths. The first path is to the 5G devices and users that the Wavelength Zone is designed to serve. This path goes through the Carrier Gateway into the carrier's mobile network. The second path is back to the parent AWS Region for access to services, data, and management functions that are not available locally in the Wavelength Zone. This path goes through the standard VPC networking to the parent Region.

The Carrier Gateway is associated with the carrier network, such as Verizon, Vodafone, or KDDI, and routes traffic between your Wavelength Zone resources and the devices on that carrier's 5G network. From the perspective of a 5G user, the traffic to and from applications hosted in a Wavelength Zone stays within the carrier's network, achieving the ultra-low latency that Wavelength Zones are designed for.

Carrier Gateways are relevant only for organizations specifically developing applications for 5G Wavelength deployments. For the vast majority of applications and developers, Carrier Gateways are not relevant.

Chapter 9: VPC Flow Logs

9.1 What are VPC Flow Logs?

VPC Flow Logs is a feature that captures metadata about the IP traffic flowing through network interfaces in your VPC. Flow Logs records do not contain the actual content of network packets, the payload data, or application-level information. Instead, they capture

metadata about each network flow, which is a set of packets sharing the same source address, destination address, source port, destination port, and protocol.

Each Flow Log record contains information including the version of the Flow Log format, the AWS account ID, the network interface identifier, the source IP address, the destination IP address, the source port, the destination port, the protocol number, the number of packets in the flow, the total number of bytes in the flow, the start and end timestamps of the aggregation interval, whether the traffic was accepted or rejected, and the log status.

The action field in each Flow Log record is particularly valuable. A record with an action of ACCEPT indicates that the associated security group rule or network access control list rule permitted the traffic. A record with an action of REJECT indicates that the traffic was blocked by a security group or network ACL rule. By analyzing Flow Log records, you can understand not only what traffic is flowing through your network but also what traffic is being blocked and why.

9.2 Where Flow Logs Can Be Enabled

You can enable Flow Logs at three different levels within your VPC.

Enabling Flow Logs at the VPC level captures traffic metadata for all network interfaces in the entire VPC. This provides the most comprehensive view but generates the largest volume of log data.

Enabling Flow Logs at the subnet level captures traffic metadata for all network interfaces in a specific subnet. This is appropriate when you want detailed visibility into traffic patterns for a specific tier of your application without capturing traffic from the entire VPC.

Enabling Flow Logs at the network interface level captures traffic metadata for a specific ENI. This is the most granular option and is useful for troubleshooting specific instances or investigating suspicious behavior on a particular interface.

9.3 Flow Log Destinations

When you create a Flow Log, you specify where the log records should be delivered.

Amazon CloudWatch Logs is the most feature-rich destination for Flow Logs. Flow Log records delivered to CloudWatch Logs can be searched using CloudWatch Logs Insights, which supports a powerful query language for analyzing log data. You can create metric filters that extract specific patterns from Flow Logs and publish them as CloudWatch metrics, which can then be used to create alarms. CloudWatch Logs stores data for configurable retention periods from one day to forever. The primary disadvantage of

CloudWatch Logs as a destination is cost. CloudWatch Logs charges for data ingestion and storage, which can be significant for high-traffic VPCs.

Amazon S3 is a cost-effective destination for high-volume Flow Logs. S3 storage costs are significantly lower than CloudWatch Logs costs for the same volume of data. Flow Logs stored in S3 can be queried using Amazon Athena, which allows you to run SQL queries against the log files without loading them into a database. Flow Logs can also be partitioned by date, making it efficient to query logs for specific time ranges. The disadvantage of S3 as a destination is that there is no built-in alerting capability. You would need additional infrastructure to monitor and alert on patterns in S3-stored logs.

Amazon Kinesis Data Firehose is the appropriate destination when you want to stream Flow Logs to an external analysis platform such as Splunk, Elasticsearch, or a custom analytics system. Firehose handles the buffering, transformation, and delivery of the log stream to the destination.

9.4 Using Flow Logs for Security and Troubleshooting

VPC Flow Logs are an invaluable tool for both security monitoring and network troubleshooting.

For troubleshooting connectivity problems, Flow Logs let you verify whether packets are actually arriving at an interface and whether they are being accepted or rejected. When an application cannot connect to a database, you might suspect a security group rule is blocking the traffic. By examining Flow Logs for the database's ENI, you can confirm whether connection attempts are arriving and whether they are being rejected. A REJECT record for the application server's IP address targeting the database port confirms the security group is blocking the connection and tells you exactly what rule change is needed.

For security monitoring, Flow Logs enable detection of several categories of suspicious activity. Port scanning produces a large number of connection attempts to many different ports from a single source IP address. In Flow Logs, this appears as many REJECT records from the same source IP with different destination ports over a short time period. Unusual outbound connection patterns can indicate that an instance has been compromised and is communicating with command and control servers. Connections to unexpected destinations or on unexpected ports warrant investigation. Data exfiltration attempts might appear as large volumes of outbound bytes to unfamiliar external IP addresses.

9.5 Flow Log Limitations

VPC Flow Logs capture traffic based on five-minute or one-minute aggregation windows, depending on your configuration. Records appear in the log destination after a delay of several minutes following the actual traffic. This delay means Flow Logs are not suitable for real-time threat detection or real-time alerting. For real-time security monitoring, you need complementary tools.

Certain types of traffic are not captured by VPC Flow Logs. Traffic to and from the instance metadata service at the address 169.254.169.254 is not logged. Traffic to the VPC DNS resolver at 169.254.169.253 is not logged. DHCP traffic is not logged. Traffic mirrored through VPC Traffic Mirroring has its own separate logging mechanism and does not appear in Flow Logs.

Flow Logs also do not capture the content of network packets, only the metadata. If you need to inspect the actual content of network traffic for deep packet inspection or application-layer analysis, you need VPC Traffic Mirroring, which is a separate feature that copies network traffic to a monitoring appliance.

Chapter 10: Complete VPC Architecture Design

10.1 Bringing It All Together

Understanding each component of VPC networking individually is necessary but insufficient. The real skill is knowing how to combine these components into a coherent architecture that meets the requirements of a production application. This chapter presents a complete VPC architecture design that incorporates all the components discussed in this guide.

10.2 Production VPC Design Principles

Before designing a specific architecture, it is useful to establish the principles that should guide every production VPC design.

Defense in depth means applying multiple layers of security controls. No single security mechanism should be the sole protection for sensitive resources. Combine public and private subnets, security groups, network ACLs, and VPC Endpoints to create multiple barriers that an attacker would need to overcome.

Least privilege applies to networking as much as it does to IAM. Every resource should have access only to the network paths it genuinely needs. Database servers should not have internet access. Application servers should not be directly reachable from the internet. Monitoring systems should have read-only access to the resources they monitor.

High availability requires distributing resources across multiple Availability Zones. Every tier of the architecture should have instances in at least two, and preferably three, Availability Zones. No single AZ failure should cause application unavailability.

Scalability planning requires designing subnets large enough to accommodate future growth. CIDR ranges cannot be changed after creation. Design for the number of resources you expect to run in the next two to three years, not just the current state.

Operational simplicity favors managed services over self-managed infrastructure where possible. NAT Gateways rather than NAT Instances, managed database services rather than databases on EC2, and VPC Endpoints rather than internet-routed access to AWS services all reduce operational complexity.

10.3 Complete Three-Tier Architecture with All Components

A complete production architecture for a web application combines all the components discussed in this guide into a coherent design.

The VPC uses the CIDR block 10.0.0.0/16, providing 65,536 addresses for the entire environment. The VPC spans three Availability Zones for maximum resilience.

The public subnet tier contains one subnet in each Availability Zone with CIDR blocks 10.0.1.0/24, 10.0.2.0/24, and 10.0.3.0/24. Each public subnet has 251 usable addresses. The public subnet route table has the local route for intra-VPC traffic and a route for 0.0.0.0/0 pointing to the Internet Gateway. Resources in the public subnets include Application Load Balancer nodes in all three AZs and one NAT Gateway in each AZ, each with its own Elastic IP address.

The private application subnet tier contains one subnet in each Availability Zone with CIDR blocks 10.0.10.0/24, 10.0.11.0/24, and 10.0.12.0/24. The private subnet route table for each AZ has the local route and a route for 0.0.0.0/0 pointing to the NAT Gateway in the same

AZ. EC2 application servers managed by an Auto Scaling Group are distributed across all three private subnets.

The database subnet tier contains one subnet in each Availability Zone with CIDR blocks 10.0.20.0/24, 10.0.21.0/24, and 10.0.22.0/24. The database subnet route table has only the local route, providing no internet access. Amazon Aurora with a primary instance in one AZ and read replicas in the other AZs is deployed in these subnets. ElastiCache for Redis is also deployed in the database subnets.

VPC Endpoints reduce traffic costs and improve security. A Gateway Endpoint for S3 is added to the private and database route tables, allowing application servers and databases to access S3 without going through NAT Gateways. Interface Endpoints for Systems Manager, Secrets Manager, and CloudWatch Logs are created in the private subnets, allowing instances to use these services without internet access, which is valuable for compliance and for instances that have no path to the internet.

VPC Flow Logs are enabled at the VPC level with records delivered to an S3 bucket. An S3 lifecycle policy moves logs older than 30 days to S3 Glacier for cost-efficient long-term retention. An Athena database and table are configured over the S3 bucket, enabling SQL queries against the Flow Logs for security analysis and troubleshooting.

On-premises connectivity is provided by a Direct Connect connection through a Direct Connect Gateway to a Virtual Private Gateway attached to the VPC. A backup Site-to-Site VPN connection provides failover connectivity if the Direct Connect circuit is unavailable. Route propagation on the private route tables automatically adds on-premises routes learned from the Virtual Private Gateway.

10.4 Real-World Case Study: Multi-Region Financial Services Platform

A financial services company called FinanceCore operates a platform for processing loan applications. Their requirements are strict. No customer personal data may leave the Asia Pacific Region. All communication must use private network paths. The system must be highly available with no single point of failure. All network traffic must be logged for compliance audits.

FinanceCore designs their architecture as follows. They deploy in the ap-south-1 Mumbai Region to satisfy the data residency requirement. They create a VPC with CIDR 10.0.0.0/16 spanning all three Availability Zones in Mumbai.

Their public subnets host Application Load Balancers that handle HTTPS traffic from loan applicants. The ALBs terminate TLS using ACM certificates. No EC2 instances are in public subnets.

Their private application subnets host the loan processing application servers in an Auto Scaling Group with instances spread across all three AZs. The application servers access the internet only for outbound calls to credit bureaus and identity verification services, through NAT Gateways in the public subnets.

Their database subnets host an Aurora PostgreSQL cluster with a primary in AZ-1a and read replicas in AZ-1b and AZ-1c. The database subnets have no internet route at all. The application servers access the database using private IP addresses through security group rules that allow only the application server security group to connect on the database port.

VPC Endpoints for S3, Systems Manager, Secrets Manager, and CloudWatch Logs eliminate internet traffic for these service interactions. The application stores loan documents in S3 through the Gateway Endpoint. Application servers retrieve database credentials from Secrets Manager through an Interface Endpoint. All systems log to CloudWatch through an Interface Endpoint.

The on-premises corporate network in the company's Mumbai office connects to the VPC via a Direct Connect connection. This allows compliance officers and auditors to access audit systems in the VPC from the corporate network without using the internet.

VPC Flow Logs are enabled at the VPC level and delivered to S3. An Athena configuration allows the compliance team to run queries to verify that no traffic has traversed unexpected paths, demonstrating compliance with the regulatory requirement for private data handling.

Summary

AWS networking is built on a hierarchy of physical and logical constructs. Regions provide geographic separation and complete independence. Availability Zones within each Region provide physical isolation within a geographic area for high availability. Local Zones and Wavelength Zones extend this infrastructure to meet specialized latency requirements. AWS Outposts brings the entire AWS infrastructure stack to on-premises locations.

Within a Region, the VPC is your isolated network environment. You define its IP address space with CIDR notation, divide it into subnets that represent different security tiers and

Availability Zones, and control traffic flow with route tables. Internet Gateways connect public subnets to the internet. NAT Gateways give private subnets outbound internet access while preventing inbound connections. Site-to-Site VPN and Direct Connect extend your network to on-premises environments.

EC2 instances connect to the network through Elastic Network Interfaces that can carry multiple IP address types. The choice between primary private IPs, secondary private IPs, public IPs, and Elastic IPs determines how instances are addressed and how reliably those addresses can be referenced.

VPC Peering and Transit Gateway solve the challenge of connecting multiple VPCs together. Peering is appropriate for simple, limited connectivity between a small number of VPCs. Transit Gateway provides a scalable hub-and-spoke model for large numbers of VPCs and on-premises networks.

VPC Endpoints keep traffic to AWS services within the AWS network, improving security, reducing costs, and enabling compliance with requirements for private data handling. Gateway Endpoints provide free access to S3 and DynamoDB. Interface Endpoints provide private access to over a hundred AWS services.

VPC Flow Logs provide visibility into the metadata of all network traffic in your VPC, enabling security monitoring, compliance auditing, and network troubleshooting. Combined with the other components of VPC networking, Flow Logs complete the picture of a secure, observable, and well-designed network infrastructure.